腾讯开源框架 Kuikly 再升级！率先适配“液态玻璃”，原生体验更极致

腾讯程序员ray 腾讯技术工程 2025年9月15日 20:25 广东 14人

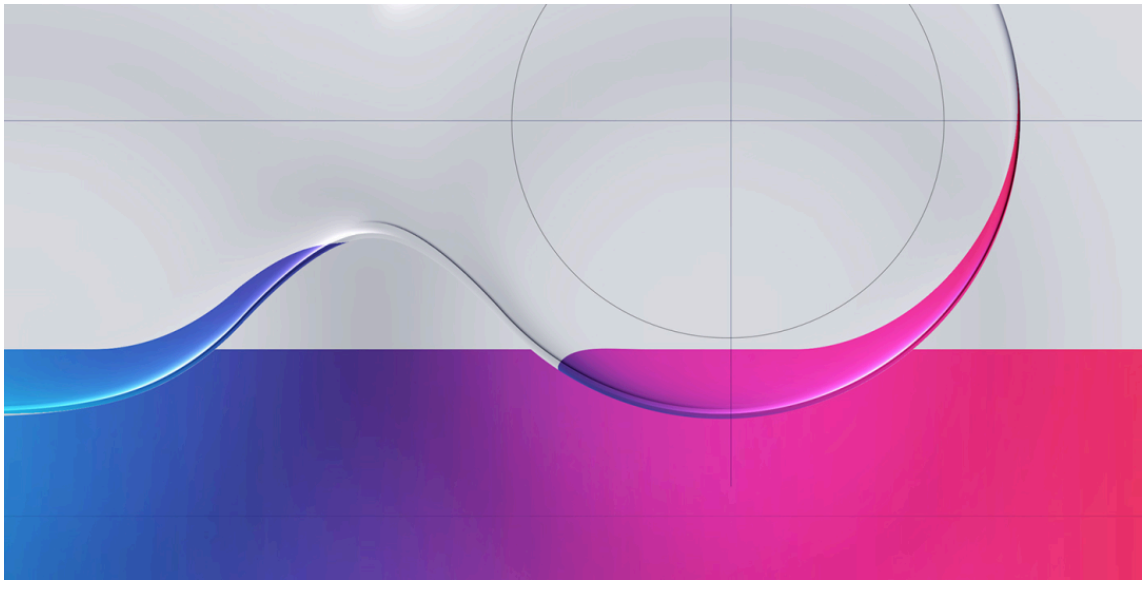


Kuikly是腾讯广泛应用的跨端开发框架，基于Kotlin Multiplatform技术构建，为开发者提供了技术栈更统一的跨端开发体验，由腾讯大前端领域 Oteam（公司级）推出。目前已有20+业务深度使用，服务业务的总页面数1000+、日活用户超5亿，满足了这些业务在众多场景下的各类复杂需求（应用场景案例）。Kuikly 作为腾讯端服务联盟(tds.qq.com)的重要成员，将持续推动跨端开发的技术创新和生态建设。

本次是在苹果发布最新iOS 26 系统的背景下，Kuikly新增全新“液态玻璃”适配，进一步丰富平台特性支持、助力业务体验提升。

一、引言

在今年的 WWDC25 上，苹果发布了其近十多年来最重要的一次视觉设计革新 —— **液态玻璃（Liquid Glass）**。这种全新的设计语言，以其半透明3D质感和动态流体效果，为用户创造了前所未有的沉浸式体验，也让软件与硬件的界限变得更加模糊。



图片1 全新液态玻璃设计

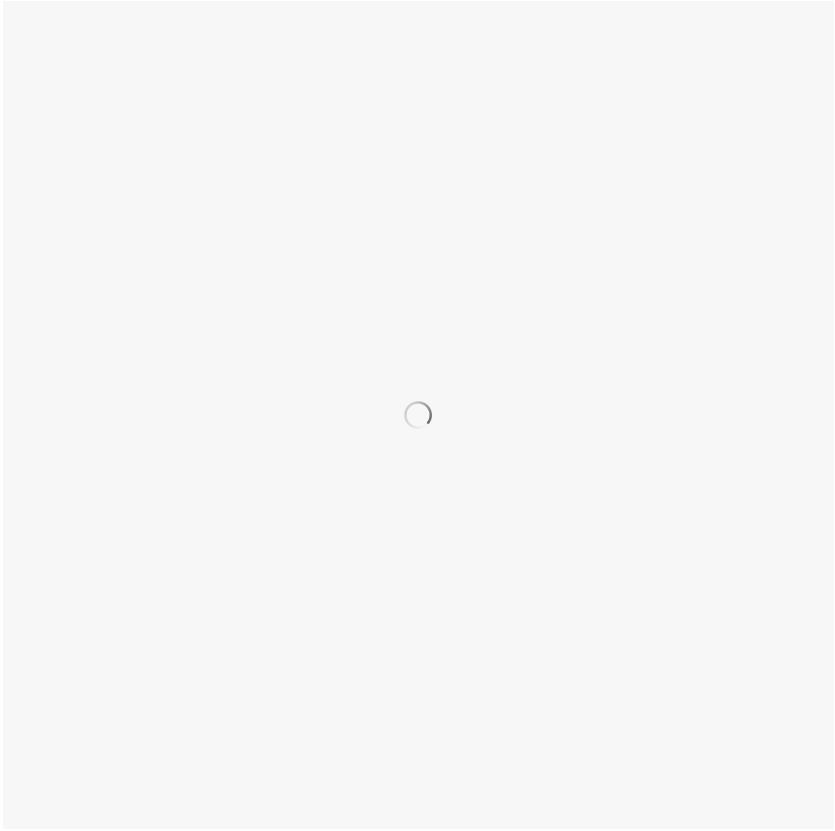
“液态玻璃”的出现，将一个跨端开发领域中长期存在的根本性问题，以一种前所未有的方式重新推回到了聚光灯下：一个跨端框架，应该如何处理与宿主系统之间的关系？

这背后是两种截然不同的架构路线：

- 自绘渲染（Self-Rendering）：追求在所有平台上提供像素级一致的体验，通过自带的渲染引擎在系统画布上绘制所有UI，从而实现最大程度的控制力和跨平台一致性。

- 原生渲染（Native-Rendering）：致力于将框架的抽象层无缝对接到原生系统的UI组件和渲染管线上，以最大化地利用平台特性、保证性能和跟进系统级的创新。

过去，关于这两种路线的讨论更多集中在性能和一致性的权衡上。而“液态玻璃”的出现，则将“ 跟进平台创新的能力 ”这一维度，提升到了一个新的高度。本文将基于这一背景，分析“液态玻璃”对跨端开发的影响，并结合 Kuikly 在“原生渲染”路线上的探索与实践，分享我们的经验与思考。



图片2 基于Kuikly实现的液态玻璃demo效果预览

二、什么是“液态玻璃”？

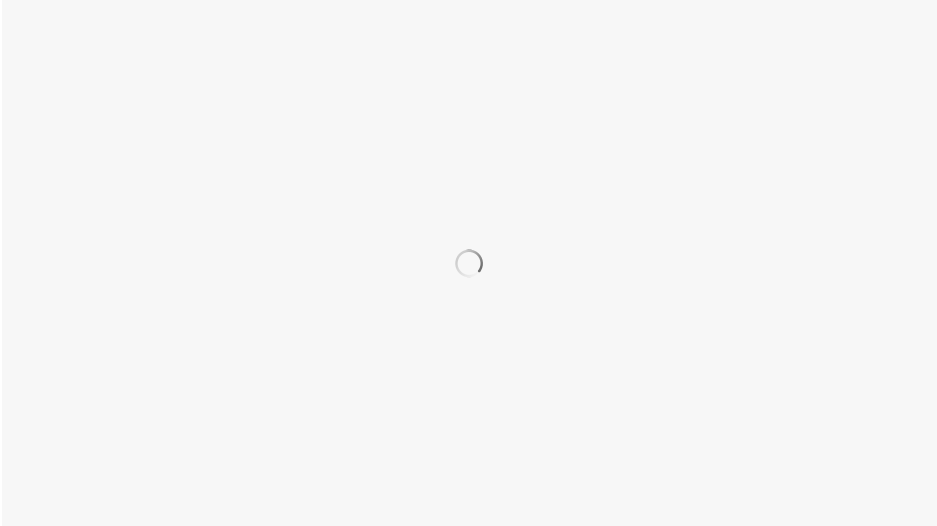
“液态玻璃”是苹果继 iOS 7 的扁平化之后，在 UI 设计上的一次重要演进，它标志着UI设计正从“扁平化”向“沉浸化”过渡，其核心在于对光学、材质和纵深感的全新探索。

从苹果给我们的解读看，这一设计的核心特征主要体现在两方面：

- **光学特性与动态流动性：**“液态玻璃”能实时根据背景内容和环境光线进行“折射”和“反射”，使UI元素的颜色和光泽随上下文动态变化。

● **多层级界面结构：** 通过将UI分为背景、内容和悬浮的互动层，创造出显著的空间感与深度。

这些特征已在最新的iOS 26系统中得到广泛应用。例如，App图标现在由多层玻璃构成，可动态适应主题色；Safari的悬浮工具条在滚动时也会自动收缩，以保证内容的全屏呈现。



图片3 苹果液态玻璃效果展示

但与以往的风格迭代不同，“液态玻璃”的实现并非纯粹的视觉技巧，而是**深度依赖于系统底层的图形处理能力**。它不再像传统设计那样模拟纸张、油墨等物理隐喻，而是通过调用硬件加速的渲染管线来直接创造光线与材质的交互效果。

这种变化意味着，UI效果的实现方式发生了根本改变：从软件层面的“模拟”，转向了对底层硬件能力的“直接调用” 。而这一转变，也给现有的跨端框架带来了新的技术挑战。

三、不同架构路线的适配分析

“液态玻璃”的推出，对应用开发者提出了新的要求。随着用户逐渐习惯系统级应用所提供的流畅、灵动的“液态玻璃”视觉效果，未适配的应用在体验上可能会出现割裂感。因此，如何有效跟进这一设计趋势，成为开发者需要考量的问题。

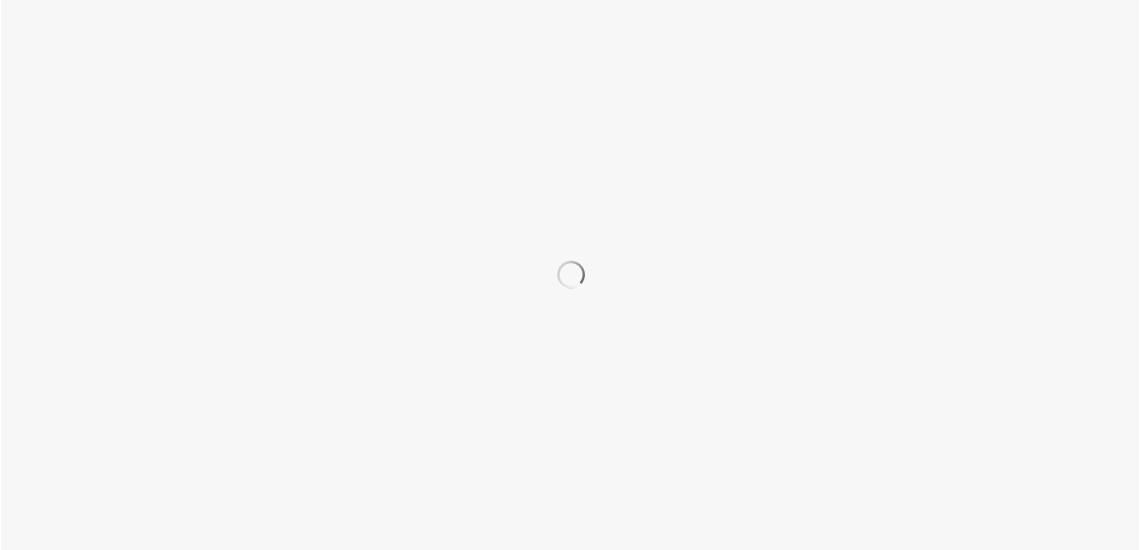
面对这一变化，基于不同架构路线的跨端框架，其适配路径和成本也表现出显著差异。

3.1自绘渲染路线

对于采用自绘渲染路线的框架（如CMP、Flutter）而言，其核心特点是在系统提供的画布上，通过自带的渲染引擎（如 Skia）绘制全部 UI。这种架构在保证跨平台视觉一致性上具备优势，但在适配“液态玻璃”这类深度依赖原生系统的特性时，则面临着独特的挑战。

因为“液态玻璃”并非简单的视觉滤镜，而是与系统渲染器（Metal）及核心动画（Core Animation）深度绑定的硬件加速效果，自绘框架无法直接调用系统 API 来实现。其适配路径通常是通过自定义着色器（Shaders）或图像处理技术，在自身渲染引擎内对原生效果进行视觉上的模拟。这种模拟不仅成本高，结果也往往不尽人意：性能和保真度较低，视觉效果难以

完全对齐原生。更重要的是，这种架构上的不匹配会成为长期问题。未来任何依赖系统级深度、光线或动态效果的更新，自绘框架都将再次面临被动的“模拟”难题。



图片4 Flutter关于如何实现液态玻璃的讨论

正如Flutter社区中部分开发者所言：“重新实现各种设计语言的成本和难度正变得越来越高。”这体现了自绘渲染路线在设计理念上的核心取舍：优先保障跨平台视觉的绝对一致性，相应地，在跟进特定平台的深度创新时则需要付出更高的工程成本。

3.2 原生渲染路线

与自绘路线不同，以Kuikly、Hippy、ReactNative等为代表的“原生渲染”框架，在适配平台级创新时则更具天然优势。这类框架通过将上层抽象映射为原生 UI 组件来进行渲染。

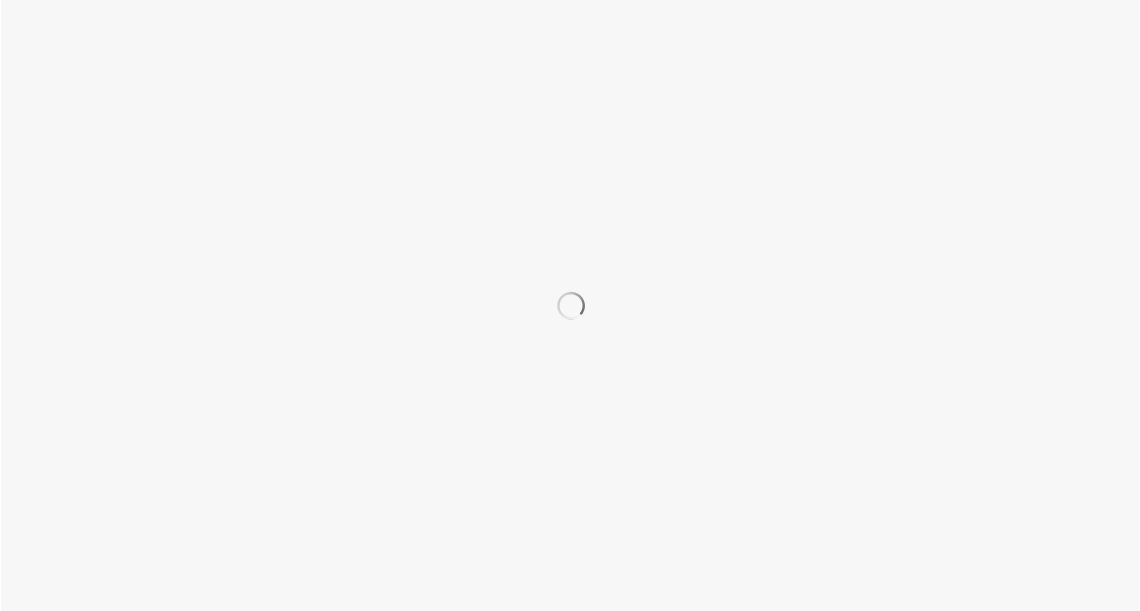
这种架构使得它们能够直接利用和封装系统发布的新特性。当苹果推出“液态玻璃”时，原生渲染框架可以通过调用系统提供的原生 API，让框架内的组件直接获得新效果的加持。

这一路径的优势在于：

- **较低的适配成本**：无需从零模拟，主要工作在于对原生 API 的封装和框架层面的暴露，开发成本相对较低。

- **较高的保真度**：由于直接使用系统能力，最终呈现的效果在视觉和性能上能与原生应用保持一致。

- **可持续的演进能力**：框架的设计理念决定了它能与宿主系统的创新保持同步。未来的平台级更新，同样可以通过相似的路径被快速集成。



图片5 Kuikly 原生渲染架构图

对于 Kuikly 而言，原生渲染的价值不止于 UI 呈现。更重要的是，这种架构选择确保了框架能够与宿主平台的技术演进保持同步，使得基于 Kuikly 的应用能够持续、低成本地集成系统级创新，给用户带来与系统原生应用相同的优质体验。

四、Kuikly的适配经验

作为致力于提供高性能、原生UI渲染的跨端框架，Kuikly现已完成对“液态玻璃”的首阶段适配，并对外开源发布。Kuikly的适配工作并非简单的UI改造，而是充分利用原生提供的基础能力，在框架渲染层和DSL驱动层两方面进行扩展，旨在为开发者提供一套便捷、低成本的适配方案。

4.1 适配原则

Kuikly框架在适配“液态玻璃”时，需要平衡一个核心问题：如何在保持我们“**通过Kotlin跨端层组合原子组件以最大化UI一致性**”的核心理念，与“液态玻璃”所强调的平台原生特色之间做出选择。面对这一挑战，我们选择灵活处理：将用户体验置于最高优先级，在追求一致性的同时，积极拥抱平台特色。

我们认为，跨端一致性的重点，是在不同平台提供符合用户习惯的优质体验，而不是追求UI像素的绝对相同。因此，我们选择以一种灵活、务实的方式来拥抱平台特色。对于常用组件，尽可能在跨端层抹平差异，提供一致的开发体验；同时通过“自定义组件”来解决系统对高级组件所做的特殊优化，例如叠加的复杂交互动画效果。这种设计思路，使得我们能够在统一代码库的基础上，仍能充分发挥平台原生体验的优势。

4.2 开发者友好的API设计

Kuikly在API设计上同样追求简洁与高效。对于常用的View、Button等组件，为了适配“液态玻璃”，我们没有引入新的独立组件，而是为现有组件提供了简洁的视图属性扩展。例如，开发者只需通过一行 glassEffectIOS() 代码，即可为任意容器视图启用液态玻璃效果。

```
View {
    attr {
        glassEffectIOS() // iOS平台将自动添加液态玻璃效果
    }
    // ... 其他子视图
}
```

这种设计的主要优势在于，它避免了开发者在UI代码中编写大量平台相关的 IF/ELSE 判断。框架会自动处理平台差异：当代码在支持的iOS平台运行时，启用液态玻璃效果；而在其他平台或旧系统版本上，则平滑降级为默认样式，确保界面表现正常。通过这种方式，Kuikly将复杂的平台差异管理进行了内部抽象，让开发者可以更专注于业务逻辑的实现。



图片6 Kuikly 液态玻璃组件示例

4.3 适配方案与技术实现

针对不同类型的组件，我们采取了差异化的适配策略：

- **基础组件**： 对基础的容器组件如View、Button，我们通过原生属性扩展的方式实现适配。同时，也提供了独立的 LiquidGlass 与 LiquidGlassContainer 组件（类似于BlurView 的用

法），满足更灵活的布局需求。

• **复杂组合组件**： 对于 Input、AlertDialog 等组合型组件，支持通过组合效果，让业务以较低成本按需适配。

• **iOS特有组件**： 对 Slider 和 Switch 这类在iOS 26上拥有全新动态效果的控件，我们在渲染层新增了iOS平台专属的组件进行封装，这确保了这些控件在具备液态玻璃效果的同时，能够获得与原生完全一致的交互体验。在上层DSL使用上，我们封装了平台差异，开发者无需修改原有组件的使用代码，只需添加 enableGlassEffect(true) 属性，即可轻松启用。

```
Switch {
    attr {
        isOn(true)
        enableGlassEffect(true) // iOS 26.0+将自动渲染为原生Switch
        thumbColor(Color.WHITE)
        onColor(Color.BLUE)
        unOnColor(Color.GRAY)
    }
    // ...
}
```

4.4 兼容性保障

在适配新特性的同时，Kuikly也全面考量了兼容性问题：

• **对旧系统的兼容**：对于使用了液态玻璃的组件，Kuikly封装的系统原生组件会自动处理兼容性，在旧系统上平滑切换为原有样式。而对于组合类组件，框架也提供了兼容性保障，业务代码无需担忧在旧系统上出现异常。

• **对其他平台的兼容**：对于使用了液态玻璃的代码，Kuikly框架保障在Android等不支持的平台上能够自动降级到默认实现，避免业务UI代码编写大量的平台判断逻辑 。利用这一机制，开发者可以在iOS上采用最新设计语言，同时无需为其他平台维护一套独立的UI，极大地降低了开发和维护成本。

篇幅所限，更多技术实现细节，欢迎访问文末Kuikly官网或Github仓库直接获取源码。

五、总结与展望

“液态玻璃”的出现，为我们提供了一个绝佳的案例，去重新审视跨端框架的两种核心路径：是“抽象并抹平平台差异”，还是“集成并利用平台能力”？

过去，跨平台开发的核心挑战是保证体验的一致性。但随着操作系统能力的不断发展——从空间计算（AR/VR）到硬件加速的AI能力——“抽象”的成本，可能正在变为错失平台最有价值的创新。一个框架的价值，不再仅取决于它能“抹平”什么，更在于它能“释放”什么。

因此，Kuikly在“液态玻璃”上的适配，并非一次简单的功能跟进，而是验证了这样一种可能：真正的跨平台，可以既不必以牺牲平台深度特性为代价，又将原生创新直接转化为自身优势。

对于开发者而言，选择一个跨端框架，不仅是工程和技术层面的权衡，更是决定了App在未来竞争中能否保持领先。我们相信，通过与原生生态的深度融合，Kuikly 能帮助开发者在不断演进的平台之上，持续创造出卓越的应用体验。

🚀 立即体验 Kuikly，加入开源社区。

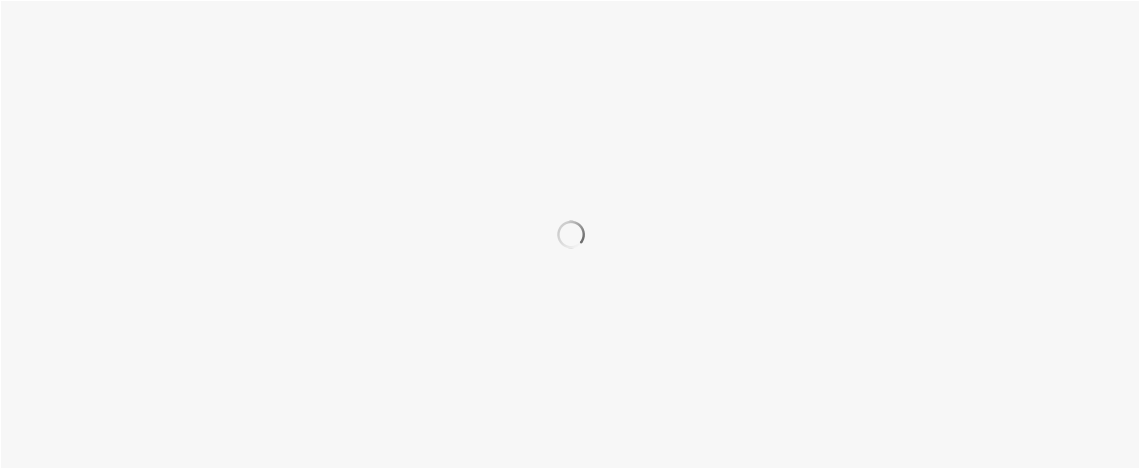
👉 **Github 仓库 | 📚 https://kuikly.tds.qq.com**

Kuikly框架属于腾讯端服务联盟重要成员，欢迎关注及了解更多信息：

腾讯端服务官网： **https://tds.qq.com/**

TDS Framework官网：

https://framework.tds.qq.com/



阅读更多：

腾讯Kuikly框架进一步开源，新增支持Web，开启一码五端新体验！

！ 福利时刻 ！

既然聊到苹果了

那今天的抽奖就选最新的AirPods Pro 3耳机



抽1位粉丝，看看谁最幸运！！！

参与抽奖方式：

点击下面卡片关注俺们公众号

腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

然后在后台回复“苹果”即可

周五下午5点准时开奖



阅读 1.2万

留言

写留言

还没有留言，来写首评